In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df=pd.read_csv(r"C:\Users\HP\OneDrive\Documents\Book2.csv")
df
```

Out[4]:

	Name	Age	Job	Earlier_Loans	Paid_Earlier_Loans	Credit_Score	Annual_Income
0	Amit	35.0	govt	2.0	Yes	720.0	850000.0
1	Riya	29.0	pvt	1.0	Yes	680.0	650000.0
2	Karan	42.0	govt	3.0	Yes	750.0	1200000.0
3	Sneha	38.0	pvt	2.0	Yes	800.0	1500000.0
4	Rahul	31.0	pvt	4.0	No	620.0	500000.0
5	Priya	27.0	pvt	3.0	No	590.0	450000.0
6	Vikas	45.0	govt	2.0	Yes	710.0	900000.0
7	Anjali	33.0	pvt	1.0	Yes	670.0	700000.0
8	Rohit	36.0	pvt	3.0	No	640.0	550000.0
9	Neha	40.0	govt	2.0	Yes	780.0	1300000.0
10	Arjun	50.0	govt	1.0	Yes	810.0	1600000.0
11	Pooja	28.0	pvt	4.0	No	600.0	480000.0
12	Manish	37.0	govt	2.0	Yes	730.0	1000000.0
13	Kavya	32.0	pvt	1.0	Yes	690.0	750000.0
14	Suresh	41.0	govt	2.0	Yes	710.0	850000.0
15	Meena	26.0	pvt	3.0	No	580.0	400000.0
16	Deepak	39.0	govt	2.0	Yes	770.0	1250000.0
17	Simran	34.0	pvt	1.0	Yes	660.0	680000.0
18	Nitin	30.0	pvt	3.0	No	630.0	520000.0
19	Alok	36.0	govt	2.0	Yes	720.0	920000.0
20	Ramesh	48.0	govt	1.0	Yes	805.0	1550000.0
21	Geeta	29.0	pvt	3.0	No	610.0	490000.0
22	Tarun	35.0	pvt	2.0	Yes	700.0	880000.0

	Name	Age	Job	Earlier_Loans	Paid_Earlier_Loans	Credit_Score	Annual_Income
23	Divya	31.0	pvt	1.0	Yes	675.0	720000.0
24	Yash	33.0	pvt	3.0	No	645.0	560000.0
25	Ankit	44.0	govt	2.0	Yes	790.0	1400000.0
26	Sunita	47.0	govt	1.0	Yes	820.0	1700000.0
27	Raj	28.0	pvt	4.0	No	605.0	470000.0
28	Isha	34.0	pvt	2.0	Yes	715.0	950000.0
29	Varun	37.0	pvt	1.0	Yes	685.0	740000.0
30	Neeraj	NaN	govt	2.0	Yes	730.0	980000.0
31	Payal	30.0	pvt	NaN	No	640.0	600000.0
32	Kunal	38.0	pvt	3.0	NaN	660.0	720000.0
33	Shreya	27.0	pvt	2.0	Yes	NaN	680000.0
34	Harsh	45.0	govt	1.0	Yes	800.0	NaN
35	Komal	32.0	pvt	2.0	No	610.0	510000.0
36	Aditya	39.0	govt	NaN	Yes	750.0	1100000.0
37	Nisha	NaN	pvt	3.0	No	620.0	530000.0
38	Sanjay	43.0	govt	2.0	Yes	765.0	1180000.0
39	Tina	25.0	pvt	1.0	Yes	655.0	540000.0
40	Mohit	34.0	pvt	3.0	No	625.0	510000.0
41	Rekha	41.0	govt	1.0	Yes	790.0	1450000.0
42	Aakash	29.0	pvt	2.0	No	615.0	495000.0
43	Bhavna	36.0	pvt	1.0	Yes	705.0	870000.0
44	Gaurav	40.0	govt	2.0	Yes	775.0	1320000.0

	Name	Age	Job	Earlier_Loans	Paid_Earlier_Loans	Credit_Score	Annual_Income
45	Sakshi	28.0	pvt	3.0	No	600.0	460000.0
46	Hemant	46.0	govt	1.0	Yes	815.0	1620000.0
47	Ritu	33.0	pvt	2.0	Yes	690.0	760000.0
48	Dev	31.0	pvt	4.0	No	635.0	525000.0

In [24]:
```
# Display first 5 rows of the dataset
print("First 5 rows of the dataset:")
print(df.head())
```

```
First 5 rows of the dataset:
    Name   Age   Job  Earlier_Loans Paid_Earlier_Loans  Credit_Score  \
0   Amit  35.0  govt            2.0                Yes         720.0
1   Riya  29.0   pvt            1.0                Yes         680.0
2  Karan  42.0  govt            3.0                Yes         750.0
3  Sneha  38.0   pvt            2.0                Yes         800.0
4  Rahul  31.0   pvt            4.0                 No         620.0

   Annual_Income  Debt_to_Income_Ratio  Loan_Amount       Loan_Type  Default
0       850000.0                  0.25      1800000       Home Loan        0
1       650000.0                  0.30       350000  Education Loan        0
2      1200000.0                  0.20      2200000       Home Loan        0
3      1500000.0                  0.15      1200000   Business Loan        0
4       500000.0                  0.40       450000   Personal Loan        1
```

In [25]:
```
# Display number of rows and columns in the dataset
print("Shape of dataset:", df.shape)
```

```
Shape of dataset: (49, 11)
```

In [26]:
```
# Print all column names
print("Columns in dataset:")
print(df.columns)
```

```
Columns in dataset:
Index(['Name', 'Age', 'Job', 'Earlier_Loans', 'Paid_Earlier_Loans',
       'Credit_Score', 'Annual_Income', 'Debt_to_Income_Ratio', 'Loan_Amount',
       'Loan_Type', 'Default'],
      dtype='object')
```

In [27]:
```
# Print datatype of each column
print("Data types of each column:")
print(df.dtypes)
```

```
Data types of each column:
Name                     object
Age                     float64
Job                      object
Earlier_Loans           float64
Paid_Earlier_Loans       object
Credit_Score            float64
Annual_Income           float64
Debt_to_Income_Ratio    float64
Loan_Amount               int64
Loan_Type                object
Default                   int64
dtype: object
```

In [28]:
```
# Display complete information about dataset
print("Dataset Info:")
print(df.info())
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49 entries, 0 to 48
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Name                  49 non-null     object
 1   Age                   47 non-null     float64
 2   Job                   49 non-null     object
 3   Earlier_Loans         47 non-null     float64
 4   Paid_Earlier_Loans    48 non-null     object
 5   Credit_Score          48 non-null     float64
 6   Annual_Income         48 non-null     float64
 7   Debt_to_Income_Ratio  48 non-null     float64
 8   Loan_Amount           49 non-null     int64
 9   Loan_Type             49 non-null     object
 10  Default               49 non-null     int64
dtypes: float64(5), int64(2), object(4)
memory usage: 4.3+ KB
None
```

In [29]:
```
# Check number of missing (null) values in each column
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
Name                    0
Age                     2
Job                     0
Earlier_Loans           2
Paid_Earlier_Loans      1
Credit_Score            1
Annual_Income           1
Debt_to_Income_Ratio    1
Loan_Amount             0
Loan_Type               0
Default                 0
dtype: int64
```

In [30]:
```
# Calculate percentage of missing values in each column
print("Missing value percentage:")
print((df.isnull().sum() / len(df)) * 100)
```

```
Missing value percentage:
Name                    0.000000
Age                     4.081633
Job                     0.000000
Earlier_Loans           4.081633
Paid_Earlier_Loans      2.040816
Credit_Score            2.040816
Annual_Income           2.040816
Debt_to_Income_Ratio    2.040816
Loan_Amount             0.000000
Loan_Type               0.000000
Default                 0.000000
dtype: float64
```

In [31]:
```
# Check if dataset contains duplicate rows
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

In [32]:
```
# Display summary statistics for numerical columns
print("Statistical Summary of Numerical Columns:")
print(df.describe())
```

```
Statistical Summary of Numerical Columns:
             Age  Earlier_Loans  Credit_Score  Annual_Income  \
count  47.000000      47.000000     48.000000   4.800000e+01
mean   35.574468       2.106383    695.625000   8.616667e+05
std     6.452989       0.937945     71.389142   3.753273e+05
min    25.000000       1.000000    580.000000   4.000000e+05
25%    30.500000       1.000000    633.750000   5.287500e+05
50%    35.000000       2.000000    690.000000   7.450000e+05
75%    40.000000       3.000000    753.750000   1.120000e+06
max    50.000000       4.000000    820.000000   1.700000e+06

       Debt_to_Income_Ratio   Loan_Amount    Default
count             48.000000  4.900000e+01  49.000000
mean               0.316458  1.058571e+06   0.326531
std                0.132913  7.829538e+05   0.473804
min                0.100000  2.500000e+05   0.000000
25%                0.200000  4.500000e+05   0.000000
50%                0.305000  7.500000e+05   0.000000
75%                0.422500  1.500000e+06   1.000000
max                0.600000  3.000000e+06   1.000000
```

In [33]:
```
# Display summary statistics for categorical columns
print("Statistical Summary of Categorical Columns:")
print(df.describe(include='object'))
```

```
Statistical Summary of Categorical Columns:
        Name  Job Paid_Earlier_Loans  Loan_Type
count     49   49                 48         49
unique    49    2                  2          5
top     Amit  pvt                Yes  Home Loan
freq       1   30                 32         11
```

In [34]:
```
# Select all categorical (object type) columns
categorical_cols = df.select_dtypes(include='object').columns

# Loop through each categorical column and display value counts
for col in categorical_cols:
```

```
    print(f"\nUnique values in {col}:")
    print(df[col].value_counts())
```

```
Unique values in Name:
Name
Amit      1
Ankit     1
Raj       1
Isha      1
Varun     1
Neeraj    1
Payal     1
Kunal     1
Shreya    1
Harsh     1
Komal     1
Aditya    1
Nisha     1
Sanjay    1
Tina      1
Mohit     1
Rekha     1
Aakash    1
Bhavna    1
Gaurav    1
Sakshi    1
Hemant    1
Ritu      1
Sunita    1
Yash      1
Riya      1
Divya     1
Karan     1
Sneha     1
Rahul     1
Priya     1
Vikas     1
Anjali    1
Rohit     1
Neha      1
Arjun     1
Pooja     1
Manish    1
Kavya     1
Suresh    1
Meena     1
Deepak    1
Simran    1
Nitin     1
Alok      1
Ramesh    1
Geeta     1
Tarun     1
Dev       1
Name: count, dtype: int64

Unique values in Job:
Job
pvt     30
govt    19
Name: count, dtype: int64

Unique values in Paid_Earlier_Loans:
```

```
Paid_Earlier_Loans
Yes    32
No     16
Name: count, dtype: int64

Unique values in Loan_Type:
Loan_Type
Home Loan         11
Education Loan    10
Personal Loan     10
Car Loan          10
Business Loan      8
Name: count, dtype: int64
```

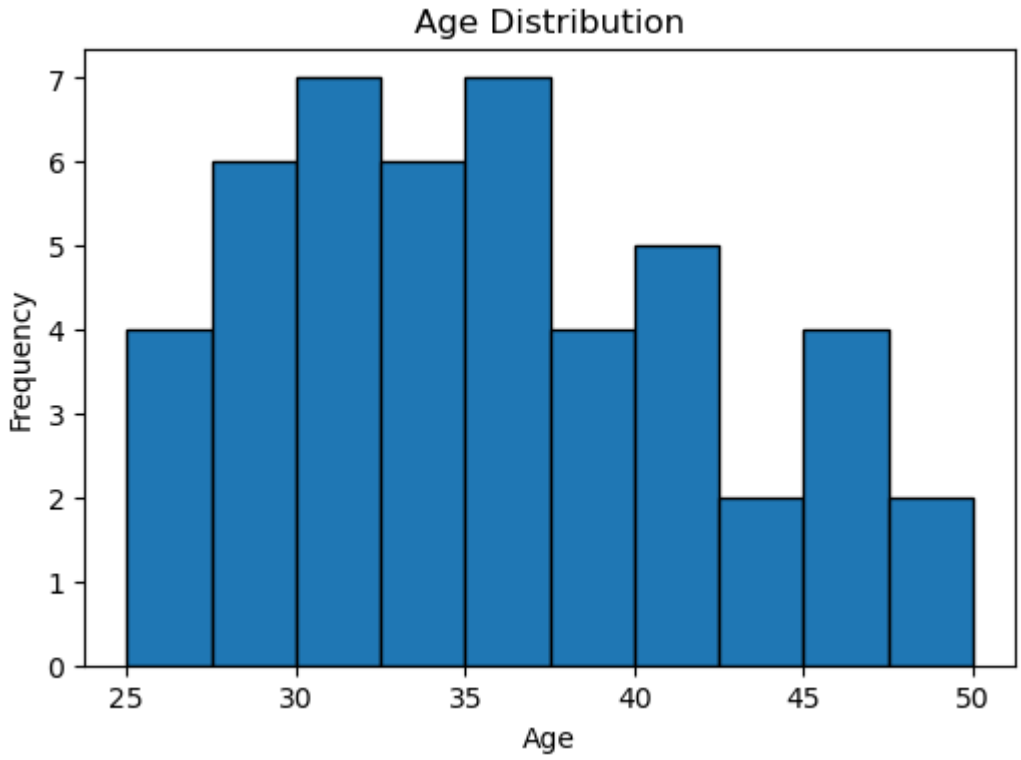
In [35]:
```
# Display count of each class in Default column
print("Default distribution:")
print(df['Default'].value_counts())
```

```
Default distribution:
Default
0    33
1    16
Name: count, dtype: int64
```

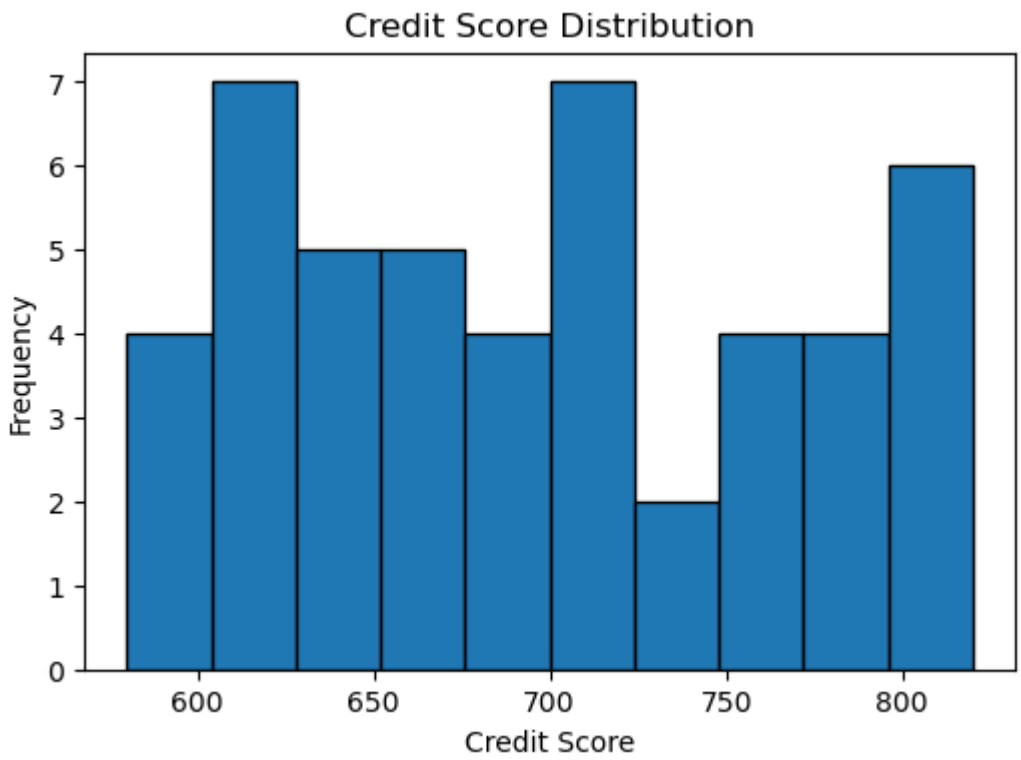
In [36]:
```
# Display percentage of each class in Default column
print("Default percentage:")
print(df['Default'].value_counts(normalize=True) * 100)
```

```
Default percentage:
Default
0    67.346939
1    32.653061
Name: proportion, dtype: float64
```

In [37]:
```
# Plot histogram for Age column
plt.figure(figsize=(6,4))
plt.hist(df['Age'].dropna(), bins=10, edgecolor='black')
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

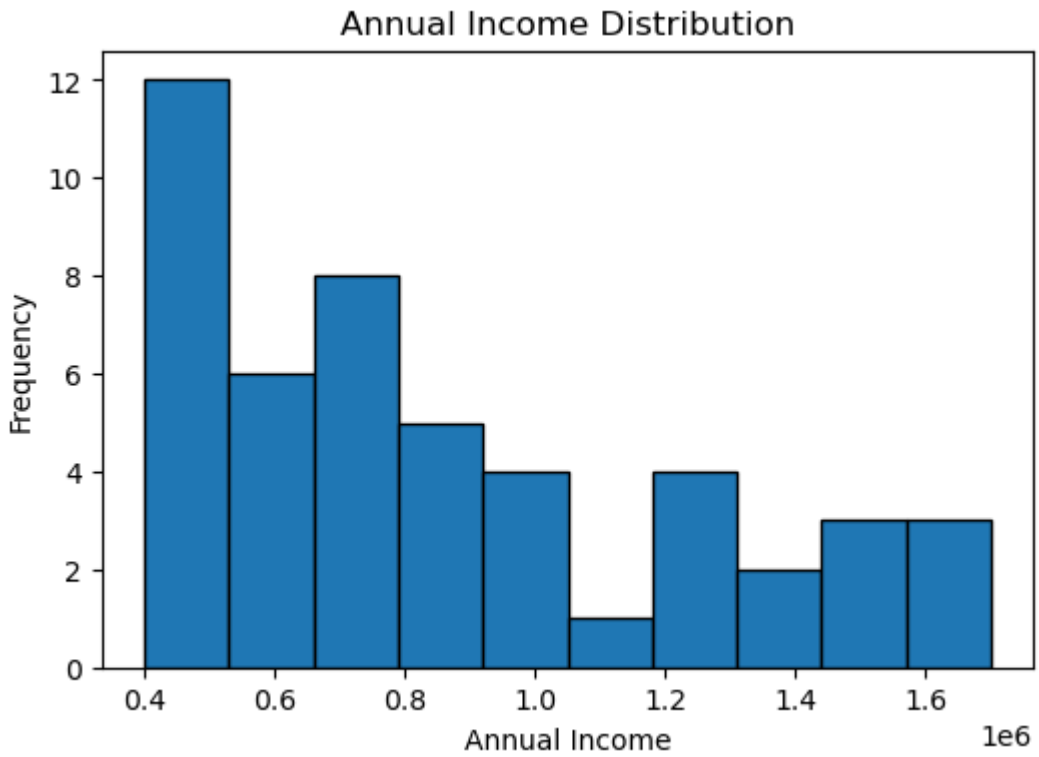


In [38]:
```
# Plot histogram for Credit Score
plt.figure(figsize=(6,4))
plt.hist(df['Credit_Score'].dropna(), bins=10, edgecolor='black')
plt.title("Credit Score Distribution")
plt.xlabel("Credit Score")
plt.ylabel("Frequency")
plt.show()
```

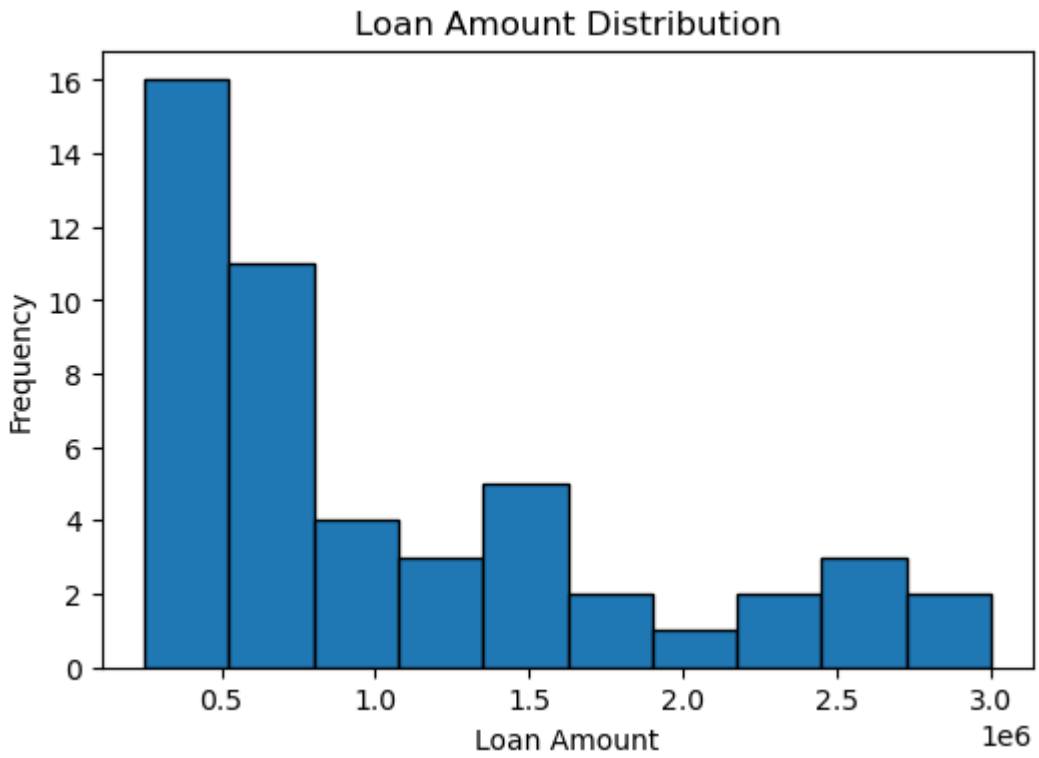


In [39]:
```
# Plot histogram for Annual Income
plt.figure(figsize=(6,4))
plt.hist(df['Annual_Income'].dropna(), bins=10, edgecolor='black')
plt.title("Annual Income Distribution")
```

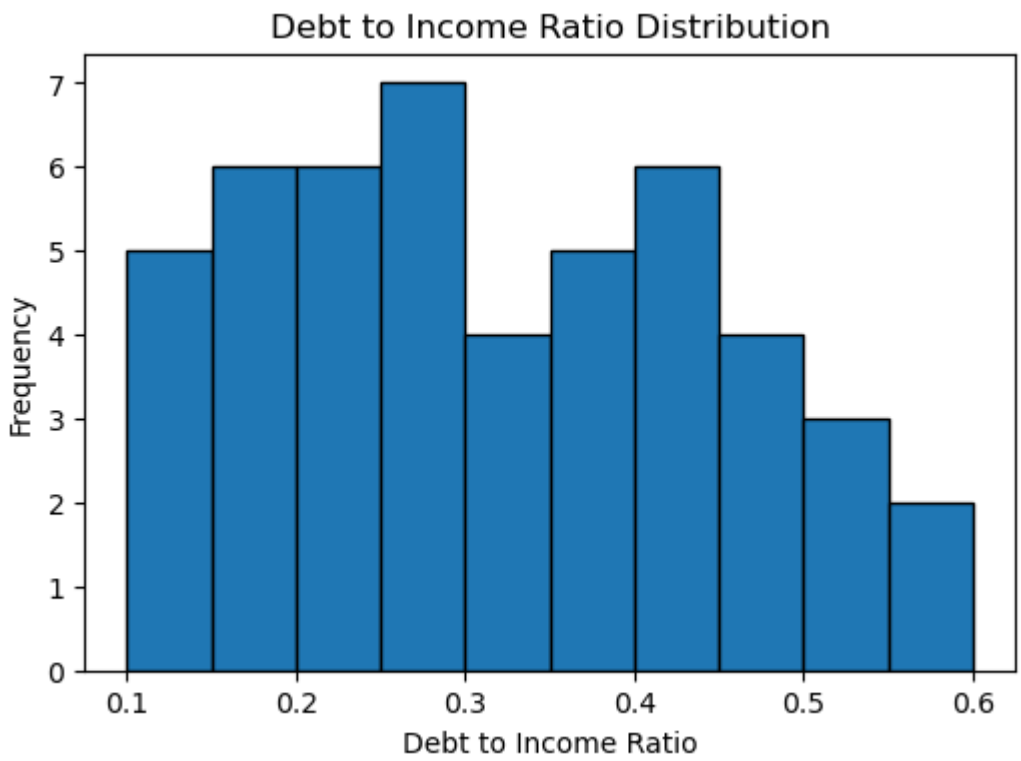
```
plt.xlabel("Annual Income")
plt.ylabel("Frequency")
plt.show()
```



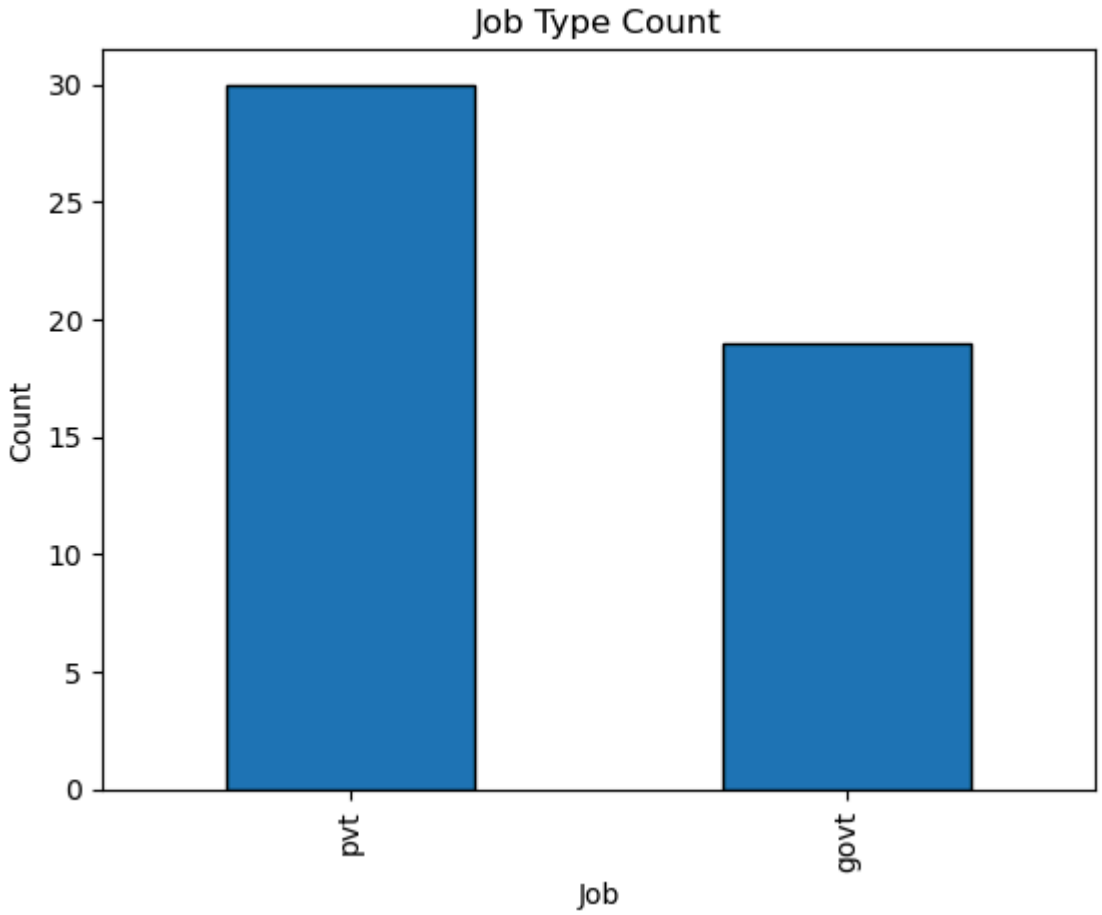
In [40]:
```
# Plot histogram for Loan Amount
plt.figure(figsize=(6,4))
plt.hist(df['Loan_Amount'].dropna(), bins=10, edgecolor='black')
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()
```



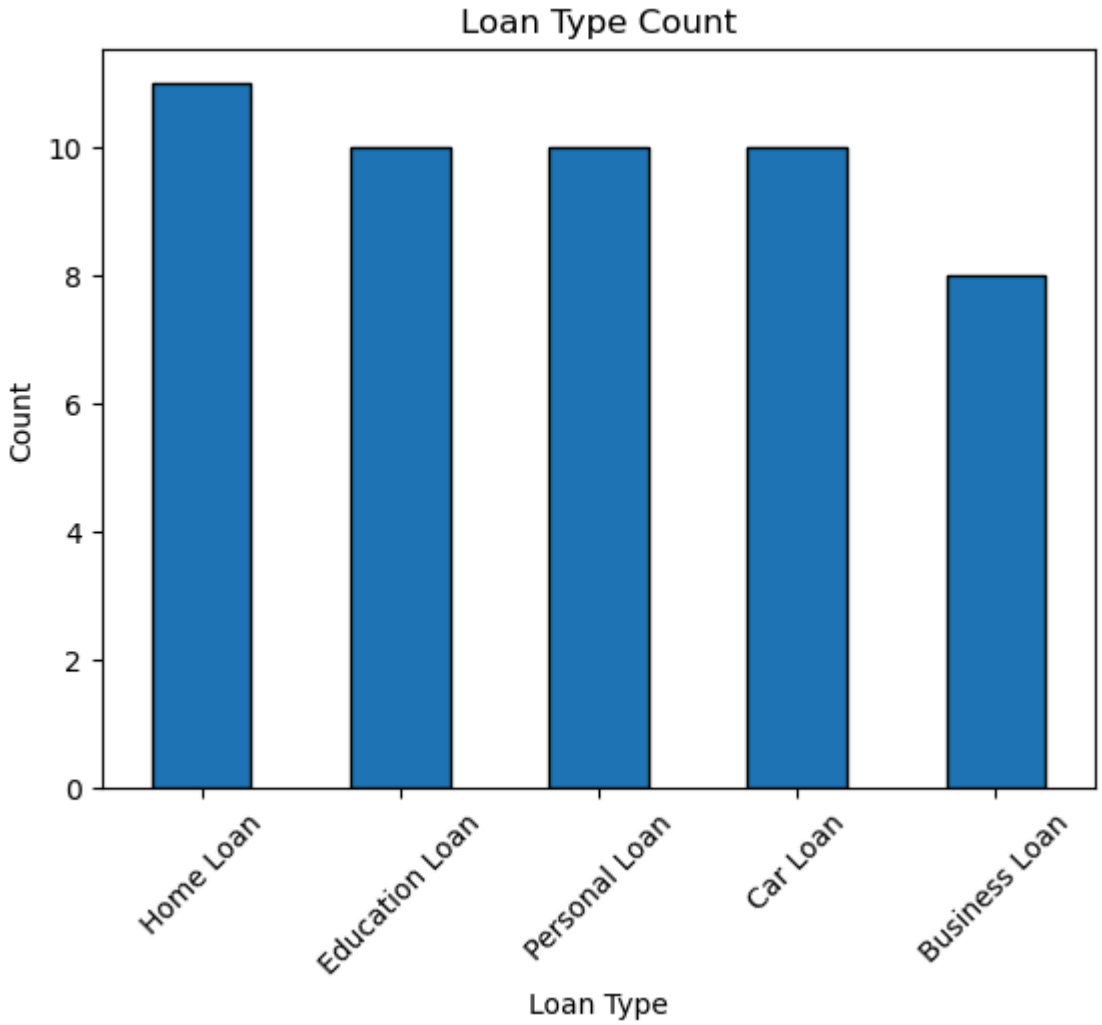
In [41]:
```
# Plot histogram for Debt-to-Income Ratio
plt.figure(figsize=(6,4))
plt.hist(df['Debt_to_Income_Ratio'].dropna(), bins=10, edgecolor='black')
plt.title("Debt to Income Ratio Distribution")
plt.xlabel("Debt to Income Ratio")
plt.ylabel("Frequency")
plt.show()
```



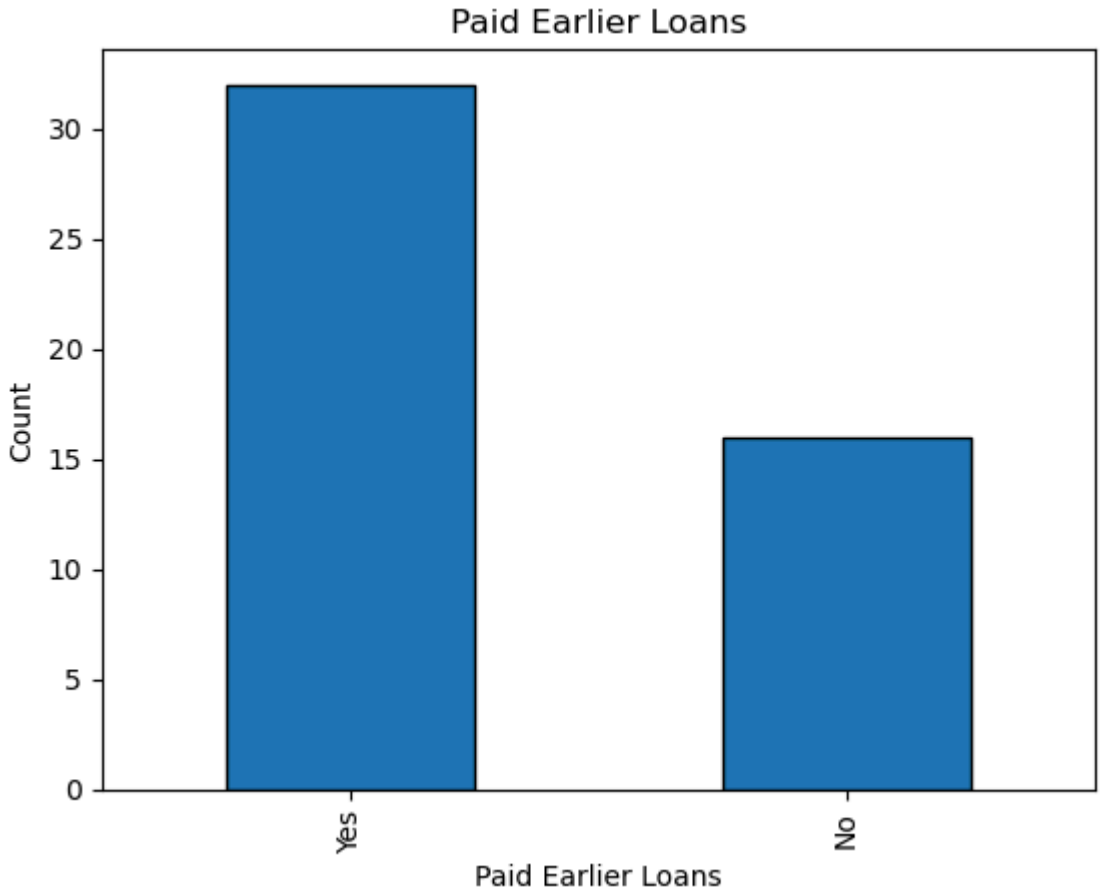
In [42]:
```
# Plot bar chart for Job types
df['Job'].value_counts().plot(kind='bar', edgecolor='black')
plt.title("Job Type Count")
plt.xlabel("Job")
plt.ylabel("Count")
plt.show()
```



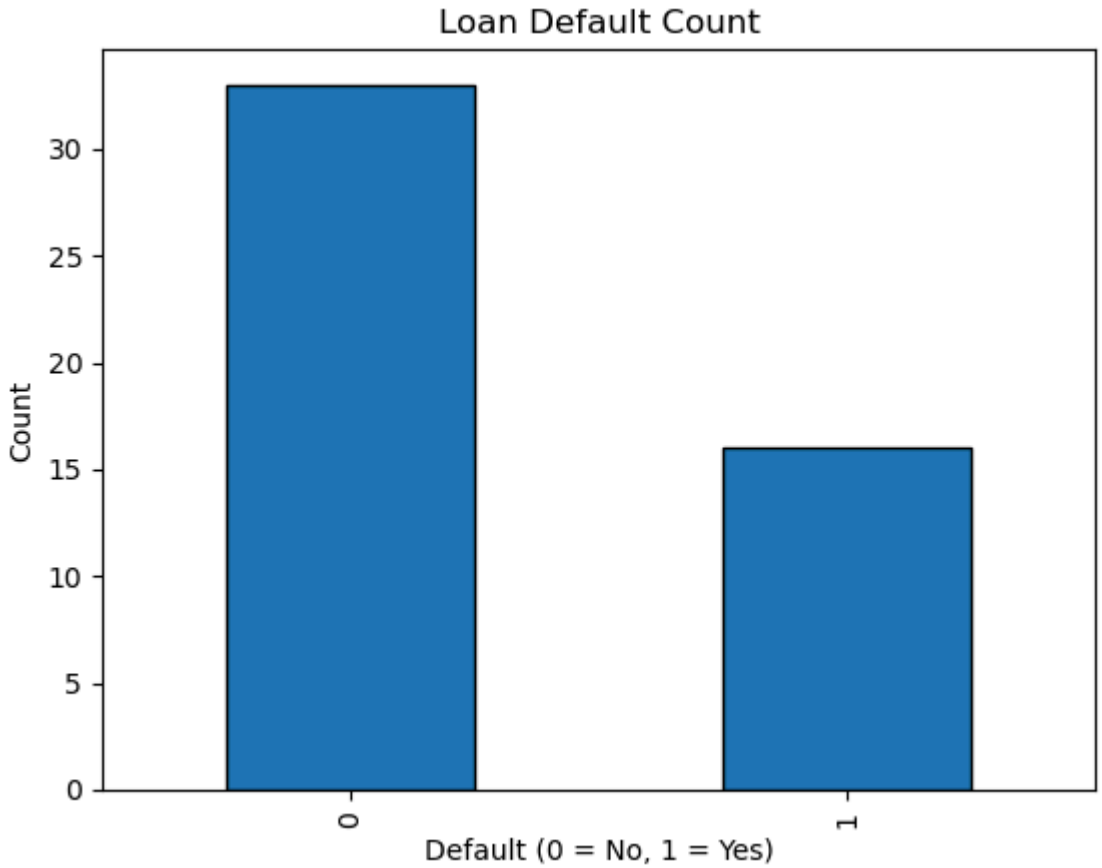
In [43]:
```
# Plot bar chart for Loan Type
df['Loan_Type'].value_counts().plot(kind='bar', edgecolor='black')
plt.title("Loan Type Count")
plt.xlabel("Loan Type")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```



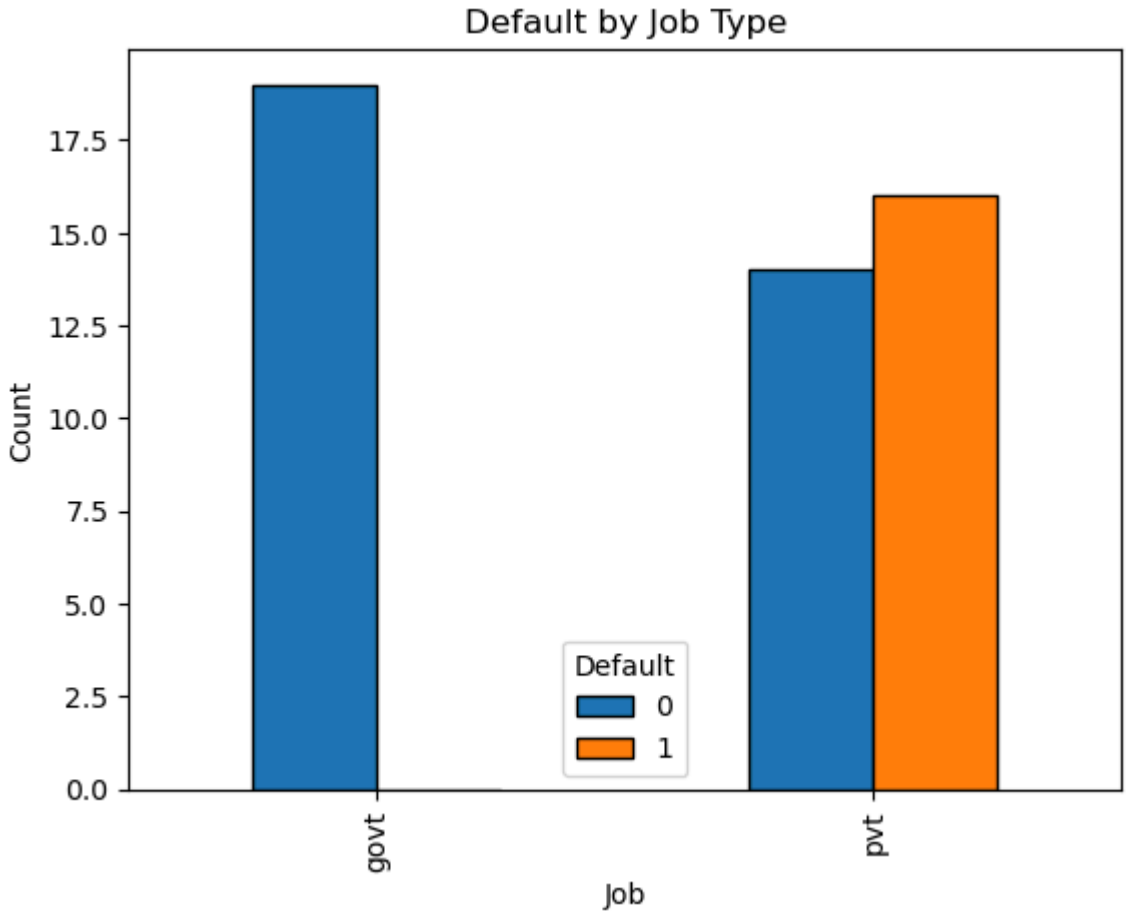
In [44]:
```
# Plot bar chart for Paid Earlier Loans
df['Paid_Earlier_Loans'].value_counts().plot(kind='bar', edgecolor='black')
plt.title("Paid Earlier Loans")
plt.xlabel("Paid Earlier Loans")
plt.ylabel("Count")
plt.show()
```



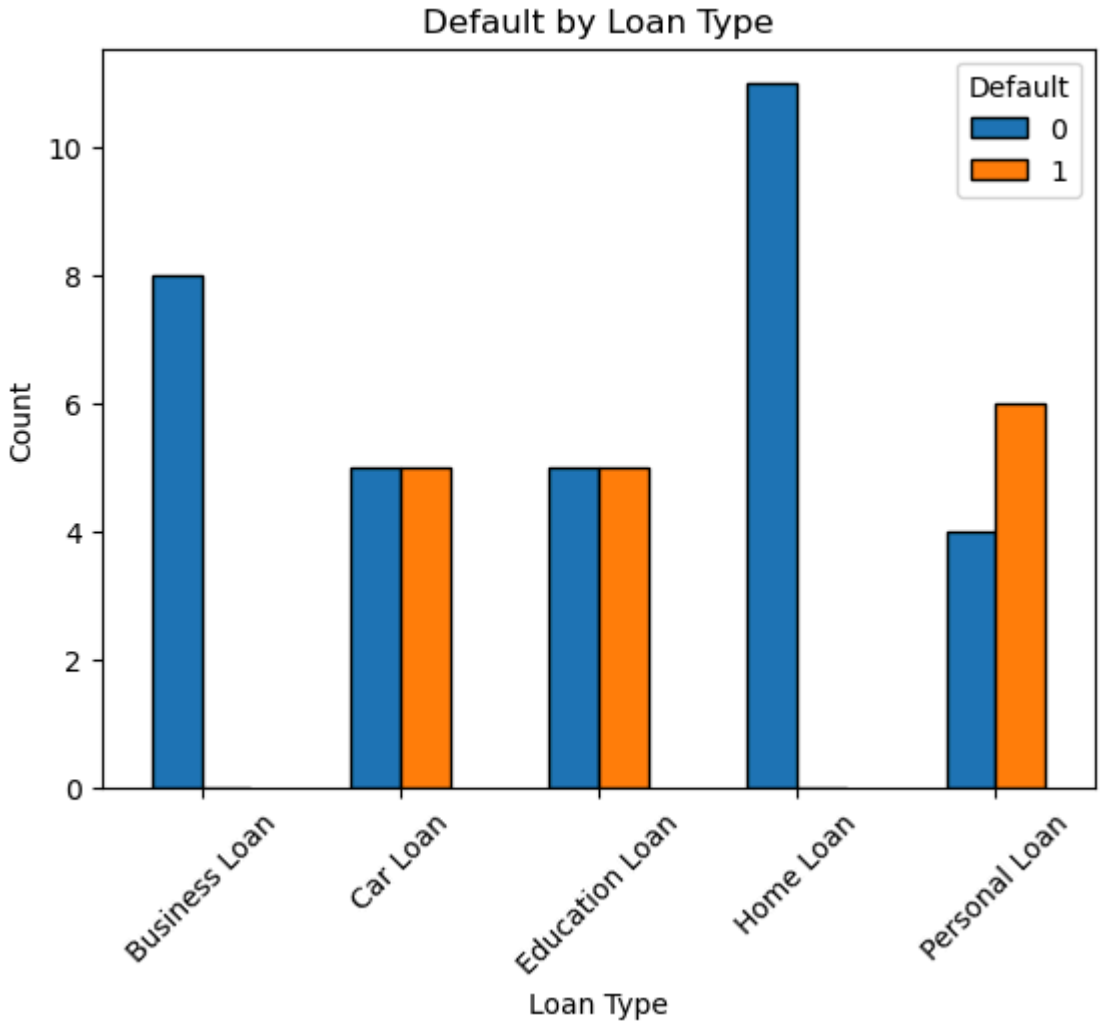
In [45]:
```
# Plot bar chart for Default column
df['Default'].value_counts().plot(kind='bar', edgecolor='black')
plt.title("Loan Default Count")
plt.xlabel("Default (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```



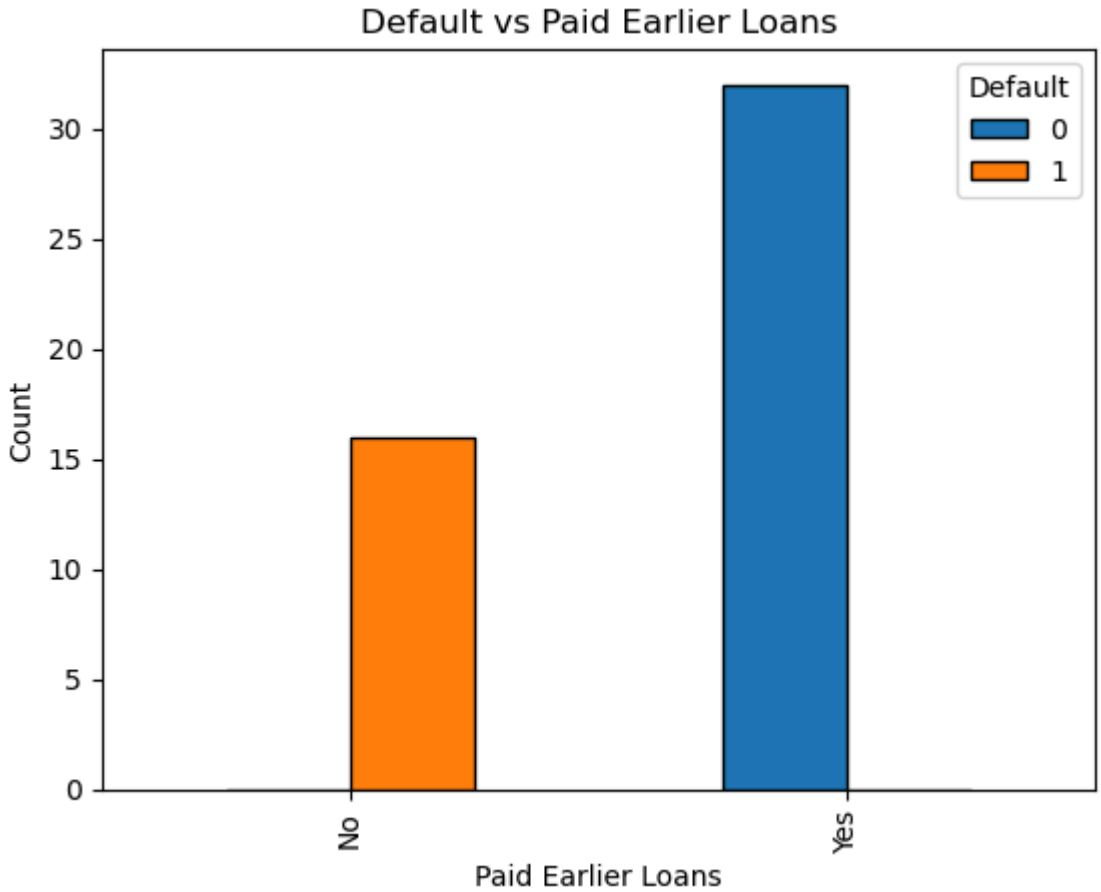
In [46]:
```
# Create a crosstab between Job and Default
pd.crosstab(df['Job'], df['Default']).plot(kind='bar', edgecolor='black')
plt.title("Default by Job Type")
plt.xlabel("Job")
plt.ylabel("Count")
plt.show()
```



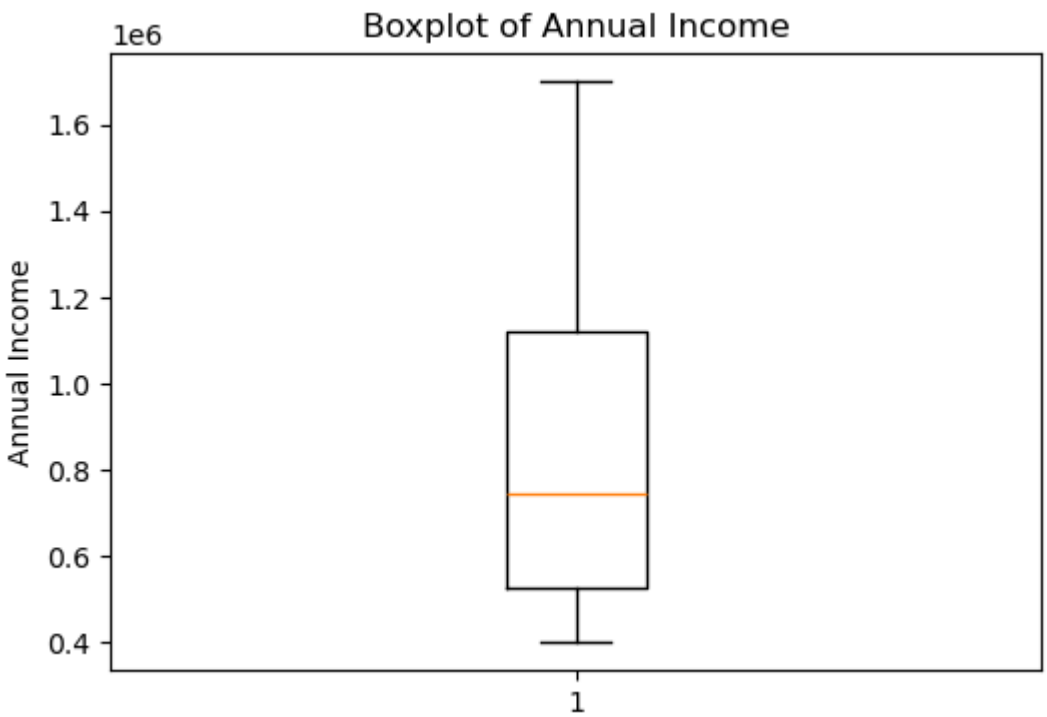
In [47]:
```
# Create a crosstab between Loan Type and Default
pd.crosstab(df['Loan_Type'], df['Default']).plot(kind='bar', edgecolor='black')
plt.title("Default by Loan Type")
plt.xlabel("Loan Type")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```



In [48]:
```
# Create a crosstab between Paid Earlier Loans and Default
pd.crosstab(df['Paid_Earlier_Loans'], df['Default']).plot(kind='bar', edgecolor=
plt.title("Default vs Paid Earlier Loans")
plt.xlabel("Paid Earlier Loans")
plt.ylabel("Count")
plt.show()
```

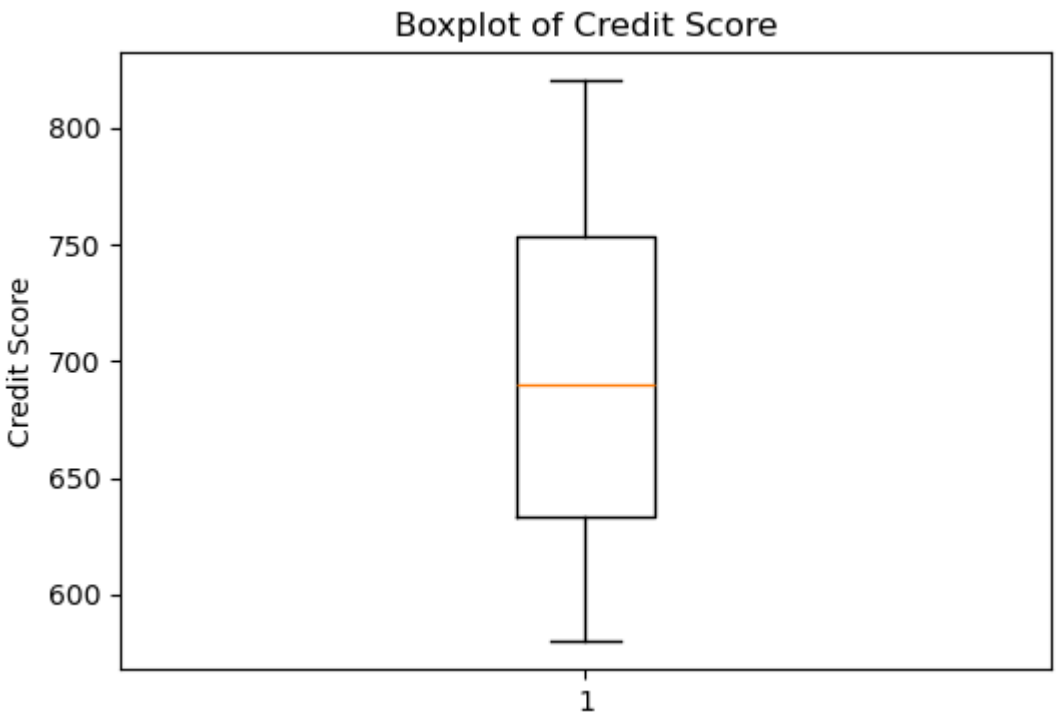


In [49]:
```
# Plot boxplot for Annual Income
plt.figure(figsize=(6,4))
plt.boxplot(df['Annual_Income'].dropna())
plt.title("Boxplot of Annual Income")
plt.ylabel("Annual Income")
plt.show()
```

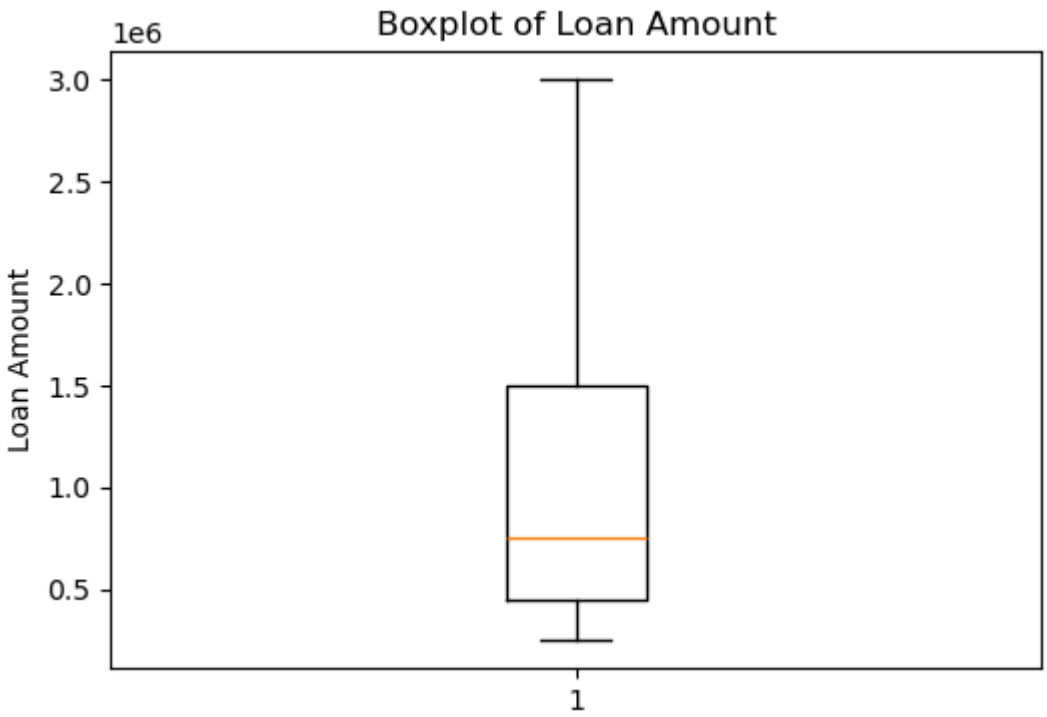


In [50]:
```
# Plot boxplot for Credit Score
plt.figure(figsize=(6,4))
```

```
plt.boxplot(df['Credit_Score'].dropna())
plt.title("Boxplot of Credit Score")
plt.ylabel("Credit Score")
plt.show()
```



In [51]:
```
# Plot boxplot for Loan Amount
plt.figure(figsize=(6,4))
plt.boxplot(df['Loan_Amount'].dropna())
plt.title("Boxplot of Loan Amount")
plt.ylabel("Loan Amount")
plt.show()
```



In [52]:
```
# Select only numerical columns
numeric_df = df.select_dtypes(include=[np.number])
```

```
# Display correlation matrix
print("Correlation Matrix:")
print(numeric_df.corr())
```

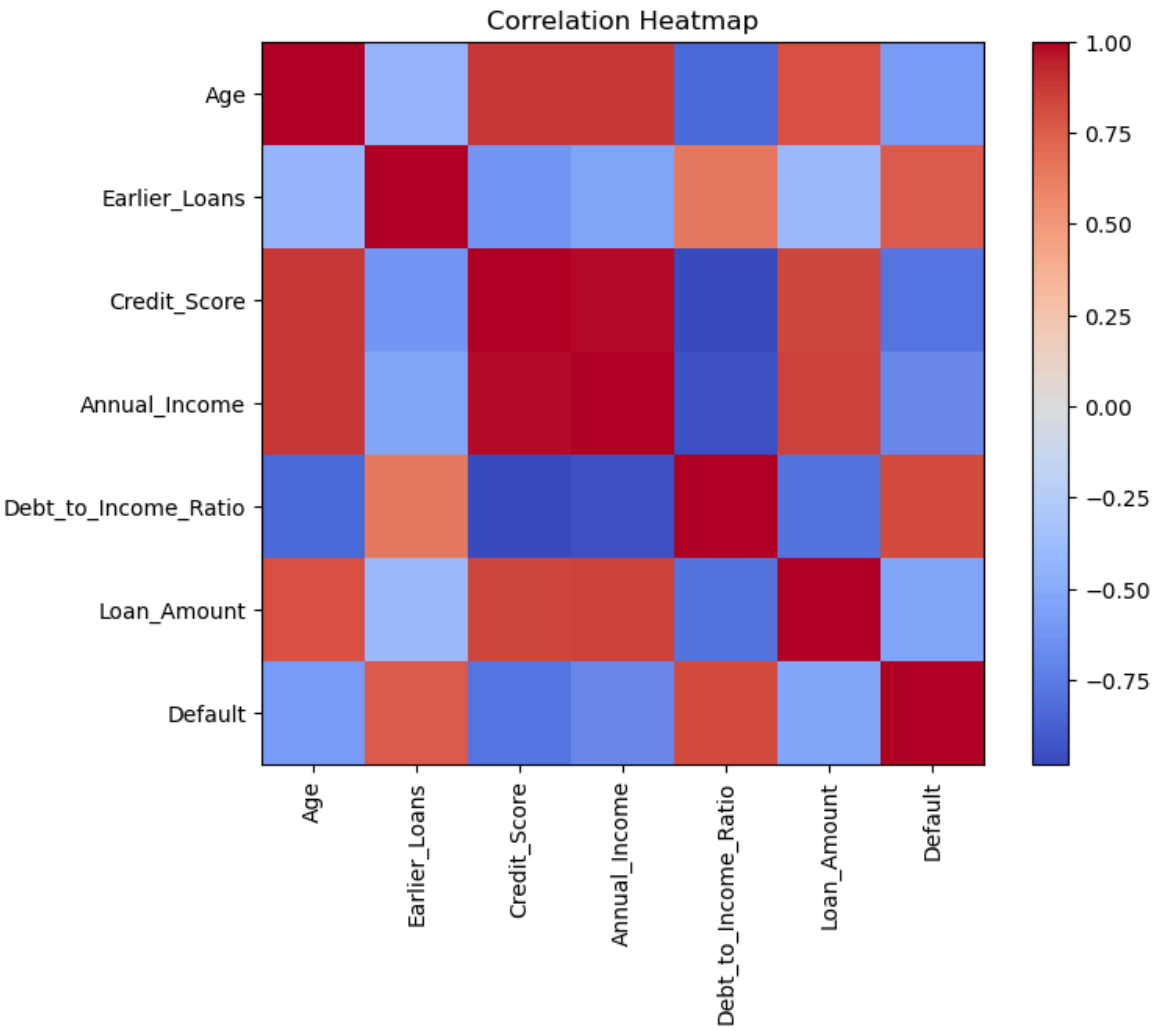
```
Correlation Matrix:
                          Age  Earlier_Loans  Credit_Score  Annual_Income  \
Age                  1.000000      -0.431207      0.880480       0.878117
Earlier_Loans       -0.431207       1.000000     -0.607880      -0.526486
Credit_Score         0.880480      -0.607880      1.000000       0.977713
Annual_Income        0.878117      -0.526486      0.977713       1.000000
Debt_to_Income_Ratio -0.848648       0.642123     -0.980544      -0.935489
Loan_Amount          0.797254      -0.393081      0.840853       0.851912
Default             -0.583538       0.757727     -0.791398      -0.682631

                     Debt_to_Income_Ratio  Loan_Amount   Default
Age                             -0.848648     0.797254 -0.583538
Earlier_Loans                    0.642123    -0.393081  0.757727
Credit_Score                    -0.980544     0.840853 -0.791398
Annual_Income                   -0.935489     0.851912 -0.682631
Debt_to_Income_Ratio             1.000000    -0.801512  0.821254
Loan_Amount                     -0.801512     1.000000 -0.521563
Default                          0.821254    -0.521563  1.000000
```

In [53]:

```
# Store correlation matrix in variable
corr = numeric_df.corr()

# Plot heatmap using matplotlib
plt.figure(figsize=(8,6))
plt.imshow(corr, cmap='coolwarm', interpolation='none')
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Heatmap")
plt.show()
```



In [54]:
```
# Find average annual income for each job type
print("Average Income by Job:")
print(df.groupby('Job')['Annual_Income'].mean())
```

```
Average Income by Job:
Job
govt    1.231667e+06
pvt     6.396667e+05
Name: Annual_Income, dtype: float64
```

In [55]:
```
# Find average credit score for defaulters and non-defaulters
print("Average Credit Score by Default:")
print(df.groupby('Default')['Credit_Score'].mean())
```

```
Average Credit Score by Default:
Default
0    735.15625
1    616.56250
Name: Credit_Score, dtype: float64
```

In [56]:
```
# Find average loan amount for each loan type
print("Average Loan Amount by Loan Type:")
print(df.groupby('Loan_Type')['Loan_Amount'].mean())
```

```
Average Loan Amount by Loan Type:
Loan_Type
Business Loan     1.278750e+06
Car Loan          7.820000e+05
Education Loan    3.580000e+05
Home Loan         2.304545e+06
Personal Loan     4.890000e+05
Name: Loan_Amount, dtype: float64
```

In [57]:
```
# Find average debt-to-income ratio for defaulters and non-defaulters
print("Average Debt-to-Income Ratio by Default:")
print(df.groupby('Default')['Debt_to_Income_Ratio'].mean())
```

```
Average Debt-to-Income Ratio by Default:
Default
0    0.243636
1    0.476667
Name: Debt_to_Income_Ratio, dtype: float64
```

In [58]:
```
# Compare important numerical features based on Default value
print("Average values grouped by Default:")
print(df.groupby('Default')[['Age', 'Credit_Score', 'Annual_Income', 'Debt_to_In
```

```
Average values grouped by Default:
               Age  Credit_Score  Annual_Income  Debt_to_Income_Ratio  \
Default
0        38.125000     735.15625      1040937.5              0.243636
1        30.133333     616.56250       503125.0              0.476667

         Loan_Amount
Default
0          1340000.0
1           478125.0
```

In [59]:
```
# Conclusion:
# 1. The dataset contains both numerical and categorical columns.
# 2. Some columns contain missing values, such as Age, Earlier_Loans,
#    Credit_Score, Annual_Income, and Debt_to_Income_Ratio.
# 3. The target column 'Default' indicates whether a customer defaulted or not.
# 4. Customers with lower credit score, lower income, higher debt-to-income rati
#    and unpaid earlier loans are more likely to default.
# 5. Loan type and job type also influence loan repayment behavior.
# 6. Some numerical columns may contain outliers, especially Annual_Income and L
# 7. Correlation analysis helps identify important factors affecting default ris
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: